introducción al paradigma

Un programa según el **paradigma funcional** será una función, una transformación de un input y un output. El output dependerá del input que se ingrese

Unicidad de funciones en programación

```
program Prueba
var
   flag: BOOLEAN

function f (n: INTEGER): INTEGER
begin
   flag ← NOT flag
   if flag then
      ↑ n
   else
      ↑ 2 * n
   end if
end
```

Cuando evaluamos el programa principal nos llevamos algunas sorpresas...

```
--- Programa Principal
begin
   flag ← true;
   ...
   write( f(1) );     ← escribe 2
   write( f(1) );     ← escribe 1
   ...
   write( f(1) + f(2) );  ← escribe 4
   write( f(2) + f(1) );  ← escribe 5
   ...
end
```

F no es matemáticamente una función, para un mismo input tengo dos resultados posibles

Transparencia referencial

$\mathrm{f}(1)$ es una expresión E cuyo resultado da un valor V.

Decimos que una solución tiene transparencia referencial si podemos reemplazar en un programa todas las expresiones E por el valor V sin alterar el resultado del programa, independientemente del lugar donde aparezca E (no depende del contexto de ejecución, ni el orden de evaluación de dicha expresión)

Variable

Mientras que una variable en un paradigma imperativo representa una posición de memoria donde almaceno valores, y me sirve para recordar estados intermedios. Una variable en el paradigma funcional respeta la **definición matemática de variable**, como una incógnita, un valor desconocido o que todavía no fue calculado.

$$f(x) = y = log\ x$$

Tanto **y** como **x** son variables matemáticas:

- Y es variable dependiente
- X es variable independiente

Valores y tipos

```
> 2 + 3
5     -- ok, la suma funciona
> not True
False
> length "Hola"
4
```

En la primera expresión `> 2 + 3` tenemos dos valores: 2 y 3, dos números y un operador, la suma que se denota + y donde se utiliza la notación *infija, para mayor comodidad (el operador se ubica entre medio de los valores). El resultado es 5, un numero*

*Otros operadores: == (comparación por igualdad), - (resta), / (división), * (multiplicación), < (menor que), > (mayor que),* ^ (elevado a), !! (desplazamientos), etc.

En el segundo caso: `> not True` *not* es una función, por lo tanto, se utiliza la notación *prefija* por defecto: primero se escribe el nombre de la función y luego los parámetros.

Ture representa al valor booleano verdadero. El resultado de evaluar esta expresión es False

Por último, en `> length "Hola"` *Length es una función, que puede calcular la longitud. En este caso de un String, o lista de caracteres. El resultado de evaluar esta expresión es 4, un número (Num).*

Modelando valores

Si nos piden que modelemos, la edad de una persona, el nombre de una materia o si un alumno aprobó un parcial.

Podemos ver qué tipo es adecuado para cada caso:

- Edad de una persona → Es un número, sin decimales
- El nombre de una materia → Es un string
- Si un alumno aprobó un parcial → Podría ser un booleano o una función

Función: Saber si un alumno aprobó

La función recibe una nota, que lo representamos como un número de a 10 sin decimales, por lo tanto:

- Recibimos un número entero sin decimales
- Esperamos como retorno un valor booleano (Aprobó / No aprobó)

Entonces definimos el tipo de función

```
aproboAlumno :: Int -> Bool
```

Y luego escribimos el cuerpo de la función, que como criterio pide que el alumno saque 6 o más para poder aprobar:

```
aproboAlumno nota = nota >= 6
```

Nota es una variable, porque en el momento de definir la función no conozco su valor, es una incógnita.

Estas dos líneas se escriben en un editor de texto, y se guarda el archivo con extensión ***.hs***

Aplicación

Para probar la función que acabamos de construir, levantamos el GHCi y desde la consola cargamos el archivo:

```
$ ghci (o stack ghci)
GHCi, version 7.10.3: http://www.haskell.org/ghc/  :? for help
Prelude> :l alumnos.hs
[1 of 1] Compiling Main             ( alumnos.hs, interpreted )
Ok, modules loaded: Main.
```

Entonces podemos probar nuestra función, **aplicando la función aproboAlumno con diferentes parámetros**:

```
*Main> aproboAlumno 8
True
*Main> aproboAlumno 3
False
```

Al pasarle parámetros a una función, se evalúa la expresión hasta obtener un valor que es irreductible para Haskell.

Consulta de tipos

Podemos preguntas cual es el tipo de una de las siguientes expresiones:

```
ghci> :t aproboAlumno
aproboAlumno :: Int -> Bool
ghci> :t 8
8 :: Num a => a
ghci> :t True
True :: Bool
ghci> :t aproboAlumno 8
aproboAlumno 8 :: Bool
```

Variables asociadas a valores

Queremos modelar la edad del señor Burns: 120

```
edadSeniorBurns = 120

*Main> :t edadSeniorBurns
edadSeniorBurns :: Integer
```

edadSeniorBurns es una variable que se asocia al valor 120.

Definiciones con guardas

En matemática podemos definir una función por partes. Para ciertos valores del dominio mi conjunto imagen era uno y para otros valores mi conjunto imagen era otro. Un ejemplo es la función max:

$$\text{max} : \mathbb{Z}^2 \to \mathbb{Z} \quad \begin{cases} \text{max } (x,y) = x & x > y \\ \text{max } (x,y) = y & y >= x \end{cases}$$

En Haskell esto se traduce:

```
maxTest :: Int -> Int -> Int
maxTest x y | x > y = x
            | otherwise = y
```

Segunda función o el uso innecesario de guardas

Queremos modelar la función que nos dice si podemos avanzar nuestro auto en base a la indicación del semáforo

- Rojo o amarillo implica que no avanzaremos
- Verde nos permite avanzar

Nuestro input es “rojo”, ”amarillo” o ”verde”. El resultado de esta función es un Bool: avanzo o no avanzo.

```
puedoAvanzar :: String -> Bool
puedoAvanzar color = color=="verde"
```

Esta función retornará True cuando color == ”verde” y False cuando color sea cualquier otro string